p0873R1
A plea for a consistent, terse and intuitive declaration syntax

Draft Proposal, 2017-11-27

This version:
https://cor3ntin.github.io/CPPProposals/unified_concept_declaration_syntax/P0873.html

Author:
Corentin jabot

Audience:
EWG, SG8

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

A unified variables and parameters declaration syntax for concepts

Table of Contents

1 Introduction

1.1 Existing declaration syntaxes in C++17

1.2 Inconsistencies and confusing syntax

2 Changes from Previous revision

2.1 Changes from P0873R0

3 Proposed Solution

3.1 Allow auto as a parameter in functions

3.2 Consistent use of auto and typename

3.3 Allow Multiple constraints for each type

3.4 Allow use of concepts consistently and uniformly.
3.4.1 Function parameters
3.4.2 Function return type
3.4.3 Variable declaration
3.4.3.1 A note on usage of concepts constraints in non-template context.
3.4.4 Dependent names
3.4.5 type-id alias declarations
3.4.6 Don’t allow Constraints everywhere
3.5 Grammar
3.5.1 Summary

4 Multiple Parameters constrained by the same concept

5 On whether `auto` should be optional.
5.1 As the type of a callable template parameter (`auto` means generic)
5.2 As the type of a return type or variable declaration (`auto` means inferred)

6 Acknowledgments and Thanks

References
Informative References

§ 1. Introduction

This paper aims to provide a uniform and intuitive syntax to declare function parameters, template parameters, lambda parameters and variables, principally with regard to the concept TS.

In particular, using function template parameters is often needlessly ceremonious, and inconsistencies have started to appear in in the language. For example, lambdas can have inferred auto parameters while functions can’t.

These factors contribute to make C++ harder to teach and encourage juniors developers to mindlessly add token until “it compiles”, regardless of whether the final program is semantically correct.

§ 1.1. Existing declaration syntaxes in C++17

Qualifiers notwithstanding, here is a list of existing way to refer to a type in declarations

Template parameters	Function parameters	Lambda parameters	Variable declaration

`template <typename|class T>`
`template <auto T>`
`template <literal T>`
	`Type`	`Type`
		`auto`
			`Type`
			`auto`

The concept proposal in its current form ([P0734R0] would add for template parameters:

```
template <constraint T>
```

§ 1.2. Inconsistencies and confusing syntax

As things stands, we notice a number of inconsistencies:

- Lambdas can be made generic with `auto` parameters, while functions can not. a side effect of that is that function template declaration is needlessly ceremonious

  ```
  template <Regular R>
  void foo(R & r);
  ```

- A single constraint can be applied to a template parameter, while variables declaration cannot be constrained.
- It is not possible to specify multiple constraints.

§ 2. Changes from Previous revision

§ 2.1. Changes from P0873R0

- Examples based on [n4685]
- More details regarding dependent names
- More details regarding alias declarations
- More details regarding `constexpr` variables
- More details regarding the meening if having more than one concept-name in a declaration

§ 3. Proposed Solution

§ 3.1. Allow auto as a parameter in functions

[P0587r0] argue against allowing `auto` in function. However, in order to be consistent with lambdas and for the sake of being terse, the declaration of `f(auto a)` should be valid.

Note that the presence of `auto` is enough to inform the reader that the function is in fact a function template.

`template<...> void f(auto a)` (as proposed by [P0587r0]) does not offer any extra information to the user of the function (nor the compiler).

Lambda now accepts either a template parameters list, auto parameters or any combination of the two, with great effect. It's a syntax that developers are already familiar with and it would only be natural for it to be used for functions too, for consistency and simplicity.

There is a wording inconsistency between `function template` and `generic lambda`. That may need to be addressed. I would suggest that `lambda template` would be a more accurate and consistent wording. Both form of lamba templates (using auto and using a parameter list) - are currently called "generic", so they are at least some consistency there, but they are a template entity behaving like a template and should therefore be refered as such.

§ 3.2. Consistent use of auto and typename

- As it is already the case, `typename` is a placeholder for a type, while `auto` is the placeholder of a value of an inferred type. This behaviour should remain consistent.
- Moreover, `typename` or `auto` should *always* be usable where they are usable today, even in the presence of a concept declaration.

However they are not required in a template parameter declaration. For example, `Even T` and `Even auto T` are unambiguous declarations and the latter can be deduced from the former. Therefore, both declarations are well-formed. A set of concepts can be defined or provided by the standard library to further constrain the nature of the non-type non-template template parameters a template accepts. `template <Pointer P>`, `template<Reference R>`, `template<Integral I>` ...

It is important to note that a given concept can not be applied to both a type and a value, so there is never any ambiguity between `typename` and `auto` in a template parameter declaration.

Notably:

- `template <Regular typename T>` and `template <Regular T>` are both valid, equivalent declarations
- `template <Even auto T>` and `template <Even T>` are both valid, equivalent declarations
- `template <Even int N>` can be used in the same fashion to restrict a literal on its value while specifying its type. In this context `int` can be thought of as a constraint on the type the value `N` can have.

§ 3.3. Allow Multiple constraints for each type

- As proposed by both [p0807r0] and [p0791r0], a declaration can be constrained by multiple concepts in conjunction, such that the following declarations are semantically equivalent.

A template instantiation must satisfy each and every constrained to be well-formed.

For example:

- `template <C1 C2 Foo>` and `template <typename Foo> requires C1<T> && C2<T>` are semantically equivalent.

[p0791r0] further suggests to allow passing concepts as parameters to templates in order to compose various logic constructs, in addition to the existing `requires` clause. However this is a separate issue that probably needs its own paper, as it is an orthogonal consideration.

I don't think however that allowing complex concept-based logic constructs in abbreviated function templates or variable declarations would lead to more readable code.

Consider `Iterable !ForwardIterable auto f(Movable&& (CopyConstructible||Swappable) foo);`

I would argue that this declaration is *less* readable. The terse, "abbreviated function template" form should handle simple cases. If a type needs to be constrained on complex requirements, then it is certainly better to either introduce a new concept for these specific requirements or to write a proper requires-clause.

So I think we should allow any number (>= 0) of concept name to preceed an type. If more than one concept-name is present, the type shall satisfies each constraints. The relation between each concept name is conjective. No other logical between concept shall be expressible without the use of a requiere clause. The idea here is not to be able to construct complexes requirements in a parameter/variable declaration expression, but to select a type or value based on several of its properties.

Imagine a function returning a sorted copy of a container. The container must be simultanously `Sortable` and `Copyable`

§ 3.4. Allow use of concepts consistently and uniformly.

§ 3.4.1. Function parameters

Of course, template functions and lambdas can be constrained.

- `void f(Sortable auto & foo)` : A template function with a constrained parameter
- `[](CopyConstructible EqualityComparable<Bar> auto & foo);` : A template lambda with a constrained parameter

In both function templates and lambda templates, the keyword auto is required to denote that the callable entity is a template and will need to be instantiated. This is explained more further in : §5.1 As the type of a callable template parameter (auto means generic) .

§ 3.4.2. Function return type

The following declaration declares a non-template function with an automatically deduced type which satisfies the `Sortable` constraint:

- `Sortable auto f();`
- `Sortable auto f();`
- `auto f() -> Sortable auto;`
- `auto f() -> Sortable;`

A program is ill-formed if the deduced type of the function does not satisfies one of the constraints declared in the return type. For the sake of sanity, specifying concept names in both the return type and the trailing return typeshould be ill-formed.

`Sortable auto f() -> Iterable auto {}; // ill-formed` `Foo auto f() -> Foo auto {}; // ill-formed`

§ 3.4.3. Variable declaration

Constraints can be used to refine the type of a variable declaration.

- `AssociativeContainer<std::string, int> auto map = getMap();`
- `Iterable AssociativeContainer<std::string, int> map = getMap();`

If a variable is `constexpr`, we can ensure its value satisfies a value concept:

- `constexpr Even int answer = 42;`
- `constexpr Even answer = 42;`

If a concept-name-list contains both type-constraining concepts and value-constraining concepts, they are checked against the type and the value respectively of the declaration to which they apply.

- `constexpr Even Integer answer = 42;`

The program is ill-formed if one of the constraint is not satisfied.

Of course, no casting or implicit conversion takes place. The left-hand side expression type is evaluated and and affected to the variables using the usual rules for `auto`, then compiler checks whether the types satisfies all the constraints introduced by each of the concept name.

As a generalization, and maybe as a mean to generate better diagnostic, I propose that constraints can be applied to any non-template or instantiated type. It would serve as a sort of static_assert ensuring that a type keeps the properties the user is expecting.

- `Iterable std::string`

§ *3.4.3.1. A note on usage of concepts constraints in non-template context.*

`Iterable vector<int>` may seem odd in first approach, but, it's a nice way to ensure that library and client code remain compatible, and to easily diagnostic what may have been modified and what why the type's expected properties changed.

It also provides documentation about what the type is going to be used for.

And most importantly, it allow to generalize the notion of constraint and make it consistent through the language.

Specifying concepts that a concrete type is expected to satisfy is not something that I expect to see often, but it could be useful in code bases that are fast changing.

§ 3.4.4. Dependent names

Constraints could be used to better document the use of a dependent names in definition of a template, instead of in addition to `typename` consider the following code

```
template <typename T>
auto constexpr f(T t) {
    return typename T::Bar(42);
}

struct S {
    using Bar = unsigned;
};

constexpr int x = f(S{});
```

We can replace `typename` by a constraint.

```
template <typename T>
auto constexpr f(T t) {
    return Unsigned T::Bar(42);
}
```

Of course, `typename` is a valid optional extra token, meaning the following construct is valid too. Note that we refer to the type `T::Bar` and not to an instance of `T::Bar`, so the valid token is `typename` and not `auto`.

```
template <typename T>
auto constexpr f(T t) {
    return Unsigned typename T::Bar(42);
}
```

Both these declarations have identical semantics `using Foo = Unsigned typename T::Bar` `using Foo = Unsigned T::Bar`

The program is ill-formed if the dependent type does not respect the constraint(s). We can, and probably should, convey the same requirements in a requires-clause. A constraint that cannot be satisfied in the body of a template instantiation causes the program to be ill-formed, SFINAE is not engaged.

`typename` when used to resolved dependent names convey little meaning to the reader, even when it's needed by the compiler. People, expecially less experienced developers, tend to add it mechanically where the compiler tell them to. [p0634r1] proposes to not requiere it in places it's not strictly needed.

In places where there are still ambiguities, using a concept name rather that `typename` add informations for the reader while still indicating to the compiler the nature of the name it is applied to - a given concept can never applies to both a type and a non-type entity.

Using concept names instead of `typename` where a suitable concept name exists to represent the dependent type would increase the readability and self-documenting properties of the program and encourage people to think about their type more

§ 3.4.5. type-id alias declarations

Allow the use of Constraints in using directives, in the same manner:

`using CopyConstructible Foo = Bar;` `template<typename T> using Swappable Foo = Bar<T>;`

In both cases, `Foo` is exactly an alias of `Bar`. However the program is ill-formed if the constraints can not be satisfied.

Additionally, it may be interesting to constraints parameters of template using-directives

`template<Copyable T> using copyable_vector<T> = std::vector<T>;` `//constrained alias on type` `template<Copyable T> using CopyableContainer<T> = Container<T>; //constrained alias on concept`

Here, the `MovableContainer` type can only be constructed using types that satisfy the `Movable` concept. We use the alias syntax to create a type alias more constrained than the original type. An alias created in this way is neither a specialization nor a new type. `copyable_vector<T>` is exactly of the type `std::vector<T>`, aka `std::is_same_v<copyable_vector<T>, std::vector<T>> == true` for any given `T`. However, the alias has additional constraints on its parameters that are checked upon instantiation. The constraits on the alias are checked before the constraints on the type it is created from.

```
copyable_vector<std::string>  ;  // Ok
copyable_vector<std::fstream> ; // Ill formed : fstream is not copyable
```

This system would allow the creation of fine-tuned constrained type alias without having to introduce new types, factories or inheritance.

§ 3.4.6. Don’t allow Constraints everywhere

There is a number of places where allowing concepts in front of a type would either make no sense or be actually harmful.

- sizeof and cast expression : It doesn’t make a lot of sense
- class/struct/union/enum definition

Specifying one or several concepts on a class definition has the result of thightly coupling a type and a concept and I feel that may be defeating the very notion of concepts. It would look more like regular inheritance that an implicit interface that exists independantly of any type.

§ 3.5. Grammar

In the situations illustrated above, the type-id can be preceded by one or more qualified-concept-name The sequence of concept-names and type form the entities to which attributes, and qualifiers are applied.

```
Swappable Sortable Writable foo = {};
const Swappable Sortable Writable & foo = {};
volatile Swappable Sortable Writable auto* foo = {};
Swappable Sortable Writable auto* const* foo = {};
Swappable Sortable Writable && foo;
extern std::EquallyComparable boost::vector<int> baz;
```

[p0791r0] gives a good justification for this order, in that constraints are like adjectives applied to a type.

A simpler explanation is that constraints are applied to a type-id. And qualifiers are applied to a constrained type. Beside the parallel with English grammar rules, putting the concept-names after the type-id would create ambiguities as to whether the constraints are applied on the type or on a value.

`auto Sortable v = /*...*/;` this would create ambiguity as to whether the type or the value is qualified by "Sortable".

Note that, as specified by [P0734R0] (§17.1), concepts can have template arguments beside the type of which we are checking the constraints.

```
Container<int> i = { /*...*/ };
Callable<int(int)> f = { /*...*/ };
```

3.5.1. Summary

- Variable declaration : `[concept-name]* type-id|auto|decltype-expression`
- Function and lambda parameters : `[concept-name]* type-id|auto|decltype-expression`
- Function and lambda return type : `[concept-name]* type-id|[auto]|decltype-expression`
- Template parameter: `[concept-name]* [typename|auto]|literal-type-id name`
- Alias declaration : `using [concept-name]* name = /*...*/;`
- Disambiguation in template definition : `[concept-name]* [typename] = /*...*/;`

4. Multiple Parameters constrained by the same concept

The meaning of `void foo(ConceptName auto, ConceptName auto)` has been the subject of numerous paper and arguments. I'm strongly in favor of considering the multiple parameters as different types. [p0464r2] exposes the arguments in favor of that semantic better that I ever could.

`void foo(ConceptName auto a, decltype(a) b)` is probably the best solution in the general case.

The following concept can be added to the stl to allow each types to have different qualifiers. In fact, the Ranges TS provides a similar utility called `CommonReference` [N4651] 7.3.5

```
template <typename T, typename U>
concept bool SameType = std::is_same_v<std::decay_t<U>, std::decay_t<T>>;

//
void f(Concept auto a, SameType<decltype(a)> auto b);
```

This syntax makes it clear that we want identic types and is, in fact, self documenting.

It is not strictly identic to the non-terse form in regards to how overloads are resolved. If we strictly want the behavior of the non-terse syntax in complex overloads sets, then the non-terse syntaxe should be used. The terse syntax aims at making the simple case simple, not at replacing all existing forms of function template declaration.

Finally, all of these reasons aside, we described how constraints could be used outside of template parameters. In all of these contexts, concepts apply to the type they immediately prefix. If functions parameters were to be treated differently that would create inconsistencies and would ultimately lead to confusion and bugs.

Beside, if the "same type" semantic is used, it won't be easy, or even possible to express that the function allow different parameter types.

§ 5. On whether `auto` should be optional.

As described above, in functions (and lambdas) parameters templates are introduced by the `auto` keyword. `auto` also lets the compiler infer the type of a declaration or a function return type.

When in presence of one or more concepts names, `auto` is not necessary in that it does not add any semantically significant information for the statement/expression to be parsed correctly and unambiguously.

`auto` may either signify generic/template parameter or deduced/inferred type depending on the context. The two scenarios should be analysed separately.

§ 5.1. As the type of a callable template parameter (`auto` means generic)

People expressed a strong desire to differentiate function templates from regular non-template functions. One of the reasons is that functions and functions templates don't behave identically.

This argument lead to the abbreviated functions being removed from the current Concept TS [P0696R1].

Given these concerns, it's reasonable to expect `auto` *not* to be optional in the declaration of a function template parameter.

`void f(ConceptName auto foo);` is a terse enough syntax, that conveys that `f` is a function template and that `foo` expect to satisfies a concept.

§ 5.2. As the type of a return type or variable declaration (`auto` means inferred)

Consider `ConceptName auto foo = /*...*/;` and `ConceptName foo = /*...*/;` Both statements are semantically identical as far as the compiler is concerned.

But, for the reader, they convey slightly different meanings.

In the first case, `ConceptName auto foo = /*...*/;` , our attention is attracted to the fact that the type of `foo` is inferred from the RHS expression, and we should probably be mindful of what that expression's type is.

In the second case `ConceptName foo = /*...*/;` however, what matters is that `foo` has the behaviour of `ConceptName`. Here `ConceptName` is seen not as a constraint of the type of `foo` but rather as a polymorphic type which exposes a known set of behaviours. The actual type of `foo` does not matter. This is why there is no need in that scenario to distinguish concepts from concrete types, as they have the same expected properties.

A developer may wish to express either of these intents and so I think that both constructs should be well-formed, and therefore `auto` should be optional when declaring a variable or return type.

It also make teaching either as both inference and concepts can be introduced independently of each other. Ultimately, some people may expect `auto` to be required while some may not. Making it optional makes the language more intuitive for all.

§ 6. Acknowledgments and Thanks

This proposal was inspired by Jakob Riedle's [p0791r0] proposal. Valuable feedbacks from Jakob Riedle, Christopher Di Bella, Michael Caisse, Arthur O'Dwyer, Adi Shavit, Simon Brand and others.

§ References

§ Informative References

[N4651]
Eric Niebler, Casey Carter. Working Draft, C++ Extensions for Ranges. URL: https://wg21.link/n4651

[N4685]
Casey Carter. Working Draft, C++ Extensions for Ranges. 20170731. URL: https://wg21.link/n4685

[P0464R2]

Tony Van Eerd, Botond Ballo. Revisiting the meaning of "foo(ConceptName,ConceptName)". URL: https://wg21.link/p0464r2

[P0587r0]

Richard Smith, James Dennett. Concepts TS revisited. 5 February 2017. URL: https://wg21.link/p0587r0

[P0634R1]

Daveed Vandevoorde, Nina Ranns. Down with `typename`!. URL: https://wg21.link/p0634r1

[P0696R1]

Tom Honermann. Remove abbreviated functions and template-introduction syntax from the Concepts TS. URL: https://wg21.link/p0696r1

[P0734R0]

Andrew Sutton. Wording Paper, C++ extensions for Concepts. URL: https://wg21.link/p0734r0

[P0791R0]

Jakob Riedle. Concepts are Adjectives, not Nouns. URL: https://wg21.link/p0791r0

[P0807R0]

Thomas Köppe. An Adjective Syntax for Concepts. URL: https://wg21.link/p0807r0